



Fundamentos de Programação

FUNÇÕES

Aula 15

Prof. Jorge Correia

Post It da Aula Passada

Outras Estruturas de Dados (Conceitos)

- Matrizes
 - Listas aninhadas
- Tuplas
 - Estáticas
- Dicionários
 - Chave/Valor

Outras Estruturas de Dados (Prática)

- Matrizes
 - Notação: `[[],...,[]]`
- Tuplas
 - Notação: `()`
- Dicionários
 - Notação: `{}`

Funções

Uma função consiste em uma **série de passos que resolve uma determinada tarefa**

Veja que uma função, na prática, nada mais é do que um algoritmo. Porém, uma função está aninhada a outro algoritmo (tipicamente ao fluxo principal do programa): **um sub algoritmo em um algoritmo**

Podemos interpretar uma função em programação como uma função matemática:

$$f(x) = 10 * x^2 + 5 * x + 1$$

Funções

$$f(x) = 10*x^2 + 5*x + 1$$

f: nome da função

(x): argumentos da função — neste caso, a variável x

$10*x^2 + 5*x + 1$: expressão algébrica da função

Quando x é definido, a função retornará um resultado, por exemplo...

$$f(1) = 10*1^2 + 5*1 + 1 = 16$$

Funções

Em Python, funções também têm **nomes**, recebem **argumentos**, executam **operações** e retornam **resultados**

Na verdade, estamos utilizando o conceito de funções desde o início da disciplina!

`print, type, len...`

E utilizamos as funções com a mesma notação da matemática:
 $\text{nome}(\text{argumento}_1, \dots, \text{argumento}_n)$

Funções

Entretanto, podemos organizar os nossos códigos por meio de funções. Ou seja, **podemos criar as nossas próprias funções!**

Essas funções podem ser definidas para...

- Organizar o fluxo principal do programa
 - Definindo tarefas específicas
 - Reduzindo blocos de código repetidos

Funções

As funções em Python são estruturadas, inicialmente, através da **definição de seu nome e argumentos (cabeçalho)**

A palavra-chave para indicar o início da definição de uma função é **def**

A seguir da palavra-chave deve constar o nome da função. A nomeação de funções segue as mesmas regras da nomeação de variáveis:

- Letras, números e *underlines* (não iniciar com números)
- Não coincidir com palavras reservadas

Funções

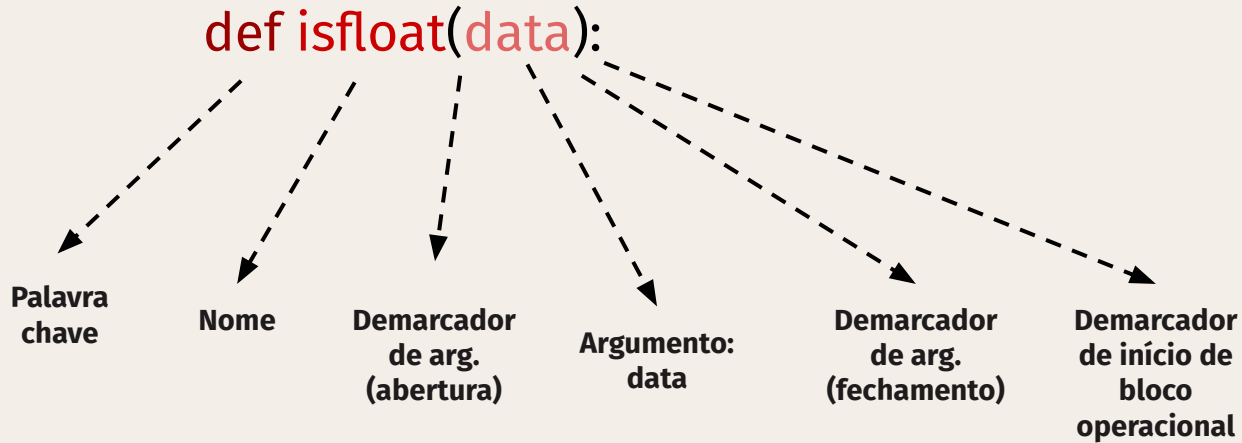
Após a definição do nome abrem-se parênteses para o estabelecimento dos argumentos esperados pela função

Os argumentos têm a mesma regra de nomeação de variáveis e devem ser **alocados de forma consecutiva e separados por vírgula**

Ao final do estabelecimento de argumentos, **fecham-se parênteses e indica-se o início do bloco operacional da função utilizando dois pontos (:)**

Funções

Assim concluimos o **cabeçalho da função**:



Funções

Na linha seguinte aos dois pontos (:) iniciaremos o **bloco operacional da função**

É absolutamente importante observarmos a **INDENTAÇÃO!**

Todo o bloco operacional deve estar indentado (hierarquizado) ao cabeçalho da função

No bloco operacional podemos ter a execução de outras funções e a definição de qualquer uma das estruturas de código estudadas até este ponto da disciplina

Funções

Sendo assim, **temos definida a nossa função** de fato:

```
def isfloat(data):  
    if data.count(".") != 1:  
        return False  
    split_data = data.split(".")  
    if not split_data[0].isnumeric():  
        return False  
    if not split_data[1].isnumeric():  
        return False  
    return True
```



Cabeçalho

**Bloco
operacional**

Funções

Mas calma aí!

O que são esses tais “*return*” aí no meio do bloco operacional?

Um *return* indica um ponto de saída de uma função:

- **Vazio**: denotado por apenas “*return*”. Em Python, envia um None (ou seja, nada) e retorna o fluxo de execução para o ponto em que a função foi chamada
- **Com dados**: denotado por “*return* dado”, onde dado pode ser uma constante ou uma variável. O dado será enviado ao ponto em que a função foi chamada, voltando o fluxo de execução para o mesmo

Funções

Com isso, podemos **evidenciar nossos pontos de saída da função exemplo:**

```
def isfloat(data):  
    if data.count(".") != 1:  
        return False  
    split_data = data.split(".")  
    if not split_data[0].isnumeric():  
        return False  
    if not split_data[1].isnumeric():  
        return False  
    return True
```

Ponto de saída #1
Valor de retorno constante: False

Ponto de saída #2
Valor de retorno constante: False

Ponto de saída #3
Valor de retorno constante: False

Ponto de saída #4
Valor de retorno constante: True

Funções

Concluída a função, podemos **a utilizar em qualquer ponto posterior à sua definição**

Para executar as operações previstas nas funções, utilizamos a mesma notação que já estamos acostumados: nome, abre parênteses, argumentos, fecha parênteses

isfloat("1.34")

Como esta função tem valor de retorno, podemos armazenar o mesmo em uma variável, por exemplo:

resultado = isfloat("1.34")

Funções

```
def isfloat(data):  
    if data.count(".") != 1:  
        return False  
    split_data = data.split(".")  
    if not split_data[0].isnumeric():  
        return False  
    if not split_data[1].isnumeric():  
        return False  
    return True
```

```
res_func = isfloat("a")  
print(res_func)
```

```
print(isfloat("1."))  
print(isfloat("a.a"))  
print(isfloat("1.a"))  
print(isfloat("10.5"))
```

Quais são os valores impressos pelas funções *print*?

De qual ponto de saída se originam esses valores de retorno em cada caso?

Funções

Naturalmente, podemos **compor funções!**

Isto é, podemos utilizar o resultado de uma função como argumento de outra função

Na verdade, acabamos de fazer isso!

```
print(isfloat("10.5"))
```


Funções

Vejamos um caso de uso de funções no escopo de bloco operacional de outra função

```
def isfloat(data):  
    if data.count(".") != 1:  
        return False  
    split_data = data.split(".")  
    if not split_data[0].isnumeric():  
        return False  
    if not split_data[1].isnumeric():  
        return False  
    return True
```

```
def tofloat(data):  
    if not isfloat(data):  
        Return False  
    return float(data)  
  
print(type(tofloat("1.a")))  
print(type(tofloat("1.4")))
```

Quais são os valores impressos pelas funções *print*?

De qual ponto de saída se originam esses valores de retorno em cada caso?

Exercício #15

Desenvolva um **programa que, em seu fluxo principal, existam duas variáveis `str_base` e `str_teste`**. Os valores de tais variáveis são definidos estaticamente em código

Devemos **verificar se a variável `str_teste` é um anagrama da `str_base`**. Porém, essa operação deve ser definida em uma **função** chamada `teste_anagrama` e ser executada no fluxo de execução principal do programa, que exibe o resultado

A função `teste_anagrama` deve ter valor de retorno ***True* caso `str_teste` seja um anagrama de `str_base` e *False* caso contrário**



Fundamentos de Programação

Aula 15

**Obrigado e
ATÉ A
PRÓXIMA!**